

FPGA Acceleration of Generative Adversarial Networks for Image Reconstruction

Dimitrios Danopoulos*, Konstantinos Anagnostopoulos†, Christoforos Kachris*, Dimitrios Soudris*

*School of Electrical and Computer Engineering, NTUA, Greece

{dimdano,kachris,dsoudris}@microlab.ntua.gr

†National Technical University of Athens, Greece

{anag.konst}@gmail.com

Abstract—Accurate and efficient Machine Learning algorithms are of vital importance to many problems, especially on classification or clustering tasks. In recent years, a new class of Machine Learning has been introduced called Generative Adversarial Network (GAN) which relies on two neural networks: a generative network (generator) and a discriminative network (discriminator). These two networks compete with each other with aim to generate new data such as images. For example, a GAN is capable of reconstructing an image which is filled by noise or has some regions damaged. Image reconstruction has found its application in the field of computer vision, augmented reality, human computer interaction and animation as well as medical imaging. However, this type of algorithm requires many MAC (multiply-accumulate) operations and high power consumption to operate. In this work, we implement an Image reconstruction algorithm with GANs, specifically as a case study we train a model capable of restoring clothing images based on the fashion-MNIST dataset. Additionally, we implement and accelerate it on a Xilinx FPGA SoC which as platforms are proven to address these kind of problems very efficiently in terms of performance and power. The design also achieves better performance and power efficiency from CPU and GPU with 0.013 *ms* average reconstruction time per image and 43 db PSNR on the FPGA quantized configuration.

Index Terms—Neural Networks, GANs, Image Reconstruction, Xilinx, FPGA

I. INTRODUCTION

In the era of big data, modern applications from the edge to the cloud require a demanding workload of many terabytes of data that are processed every day. Emerging machine learning algorithms such as Neural Networks (NNs) that are constantly learning from real-world large scale data have shown great advancement especially due to the high accuracies offered.

In recent years, there has been a new class of Machine Learning designed by Ian Goodfellow and his colleagues called GANs [1]. Two neural networks contest with each other in a game (in the form of a zero-sum game, where one agent's gain is another agent's loss). It's basically an unsupervised learning task that given some training data, learns to generate new samples with the same statistics as the training set or even reconstruct partial data such as images. Basically, there is a generator model that we train to generate new examples, and a discriminator model that tries to classify examples as either real (from the domain) or fake (generated). The applications of GANs have gained significant traction rapidly especially in the fields of Science, Video Games, Computer Vision, etc [2].

More specifically, image reconstruction which is the application we focus on this work has found its application in the field of Computer Vision and Image processing (i.e. resolution up-scaling), human computer interaction (i.e. face reconstruction) as well as medical imaging [3] (early diagnosis of partial medical images). Using state of the art models such as GANs for this kind of problems offers great accuracy and quality to the results compared with older methods.

On the other hand, the increasingly growing demands for efficient and fast processing of the new generation algorithms like CNNs or GANs in our case motivated efforts to enhance these tasks using hardware-specific optimizations by leveraging different heterogeneous architectures (i.e. CPUs, GPUs, FPGAs) [4]. Field Programmable Gate Array (FPGA) implementations have seen great advancement as is it shown that they have been extremely effective on CNN tasks due to their massive parallelism and reconfigurability on the bit level [5], [6]. This architecture shows immense parallelization and re-configurability and can be mapped well to repetitive tasks such as CNN computation which is part of GAN computation. With the utilization of hardware accelerators as we will also see in this work, low latency can be achieved as well as high total throughput. At the same time these high performance platforms achieve great power efficiency due to their architecture which is also an important factor for edge devices and cloud workload as well [3], [7].

However, GANs is a very recent but valuable topic in this field and there is little to no previous work that also combines HW acceleration. Thus, in this work, we present a novel scheme for GAN deployment on a small embedded Xilinx FPGA SoC that restores images with great quality in short time as well. In summary the main contributions are as follows:

- We develop and train a GAN model capable of generating new unseen images, using clothing images as a use case. Then, with several modifications we create a model capable of reconstructing images with very fine precision.
- We implement a HW architecture for a Xilinx Zynq 7000 FPGA for the Generator model in order to accelerate the image reconstruction algorithm with optimizations in host, memory and kernels.
- We evaluate the reconstructed images in terms of performance and quality using different bit precisions in HW and also outperform other platforms (i.e. CPU, GPU).

II. RELATED WORK

There has not been substantial previous work in the field of GANs especially using GANs with FPGAs as it is a very new subject in the research community. However, there has been several prior papers that involve image reconstruction algorithms using HW acceleration. This section will include a very similar problem domain and compare our contributions with previous work in terms of quality of results and acceleration speed.

In [8] Rajesh Mehra et al. present a tomographic image reconstruction method with reconfigurable design targeting a Xilinx Virtex 2 Pro. Specifically, they presented a reconfigurable design of filtered backprojection (FBP) for parallel beam claiming 144MHz operation frequency at almost 14% FPGA resource utilization. However, the image reconstructed which is presented in the paper has a relatively bigger quality drop vs a GAN generated image such as in this work.

S.O. Memik et al. [9] investigated an iterative image restoration for FPGA implementation. They reported several metrics like results quality or speed targeting a Xilinx FPGA. In detail, their maximum reported speed for a single image restoration of kernel execution only and using their largest FPGA is 0.28s for a 256×256 image. Even if we scale the acceleration to our image size (28×28) our design is a relatively faster.

In [10] S. Kumar develops a de-noising algorithm which can remove different types of noises especially from digital images with high accuracy based on an approximated fractional integrator (AFI) on grayscale images. Proposing a hardware implementation of this algorithm on Artix-7 FPGA they validate accuracy but no performance results are given.

Last, there has been a few prior work that involve GANs and FPGAs such as [11], [12] that present memory-efficient architectures to accelerate the generator and/or discriminative network of GANs. This is a very similar problem domain like ours however they show performance results on much larger FPGAs and no actual application is mentioned or tested. Specifically, they thoroughly describe the FPGA architecture design but no details or output image samples are given on the application side or the quality of GAN generator.

To conclude, some prior work presented image restoration algorithms on FPGA but show no matching quality and/or speed of results with our GAN-related work. Others, present a GAN-related work on FPGAs but do not include any application details or quality of generated results. In the next sections, we will present an application that has not been considered before especially for FPGA implementation which is an FPGA design of a GAN generative model for partial image restoration that also reports significant results in terms of performance.

III. SOFTWARE IMPLEMENTATION

In this section, we will describe the application design on the software level. As we already mentioned, we trained two custom MLP (multilayer perceptron) networks (one for generator and one for discriminator model) on the fashion-MNIST dataset which is slightly more challenging than the

typical MNIST. The generative model generates new image samples and with several modifications we configured the GAN model to reconstruct partial images. Also, we reduced the model parameters in a memory-efficient way to fit in the FPGA on-chip BRAMs without significant quality drop.

A. Model parameters

The discriminator consists of an MLP with a 4-layer layout with each layer consisting of a Dense, a LeakyReLU activation function and a dropout layer along with Sigmoid at the end with a total of $1.5M$ parameters. The generator model which is capable of producing the synthetic images has almost same 4-layer layout but with a Tanh output layer with a total of $1.1M$ parameters. However, as we will see next this model's parameters were remarkably reduced for HW deployment.

B. Data reconstruction technique

Throughout GAN training, the generator approximates this distribution while the discriminator tells how well it performed, and the two alternatively improve. In order to construct random samples from the distribution, the generator is given random noise as input each time. However, in the case of image reconstruction the input will be the half of an image from the dataset while the output will approximate the other half. For the generator model, the training and test set were reshaped to have only the half of the image (top half). At the end, the output half image is combined with the other half in order to feed it to the discriminator model in the training process. The training of the generator model follows the same intuition of the typical discriminator model with the binary cross entropy loss (equation 1). The aim here is to find a model that maximizes the probability of the training data.

$$H_{y'}(y) := - \sum_i (y_i' \log(y_i) + (1 - y_i') \log(1 - y_i)) \quad (1)$$

C. HW-aware training

In order to synthesize a NN model for FPGA we had to do some HW-friendly configurations in the model itself. First, the large number of parameters of the generative model ($1.1M$) needed to be reduced so as to fit into the on-chip memory of FPGA for maximum data-bandwidth. We reduced the number of layers of the network from four to three and the number of neurons as well but kept the same activation function ReLU which can be easily translated in hardware. This resulted in only $32K$ parameters which is very memory-efficient without significant degradation in results. Also, it's worth mentioning that we applied a MinMax constraint on Keras training procedure in order to keep the values of weights in a small number specifically $(-2, 2)$ (see Figure 1). This eliminates the need of large bitwidth of the multipliers and reduces resources overall. Last, as final output layer we had a Tanh function which is a bit trickier than ReLU in hardware, however we implemented it in the FPGA with a pre-stored value table. The number of output neurons in Tanh function is 392 as derived from the half image ($\frac{28 \times 28}{2}$) that is predicted.

In the following Figure, we can observe the loss achieved for discriminator and new generator model. Also, at the bottom of the Figure we can see the parameter range for each layer as acquired from the MinMax constraint we already mentioned.

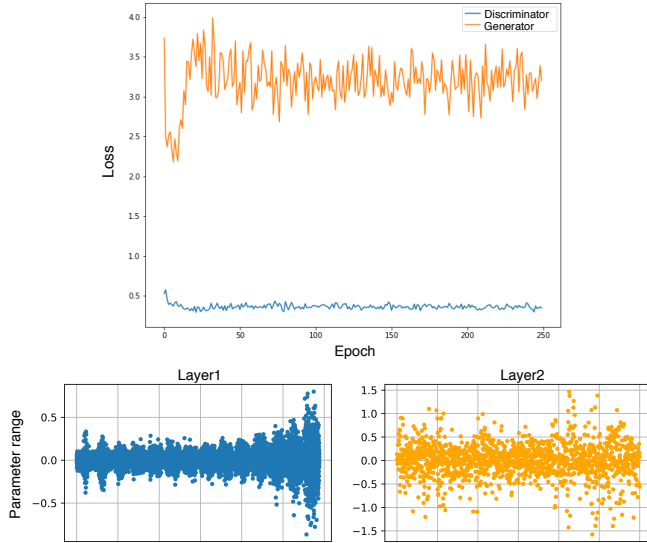


Fig. 1. Training Results (top: Loss for Discriminator and Generator model, bottom: illustration of parameter range in each layer)

In Figure 2 we can see a collection of miscellaneous images in the first epoch and last epoch of training. As it is apparent, in the first epoch the bottom half of the images (which is the output of the generator model) seems like random noise. However, in the last epoch (right) the collection of images depict complete clothing figures which resemble the real ones.



Fig. 2. Generator results for image reconstruction from first (left) and last epoch (right)

IV. ARCHITECTURE DESIGN

The aim of the FPGA acceleration was to speed-up the generator model which synthesizes/reconstructs images. In order to have a fast reconfigurable design we implemented a memory-efficient neural network as described in previous section and synthesized in hardware all its layers in a pipelined fashion. Host code optimization, buffer management regarding data exchange between the host and kernels, general pipelining on the FPGA, and synchronization between host and kernels were all precisely constructed according to FPGA design principles for high performance (Figure 3 shows final system).

A. Host optimizations

As a first step, comprise the design of the host side of the accelerator function so as to map well to the applications needs. The accelerator input is a 14×28 image which is fed through the accelerator. The input image is allocated in C++ vectors in contiguous memory so that the most efficient data movers are instantiated for the accelerator. The output of the accelerator is the predicted 14×28 image that is missing which is then transferred back to host where it is concatenated with the input to result in the final reconstructed 28×28 image.

B. Memory optimizations

Optimizing data movement and memory format is crucial to achieve high performance. With the partitioning of the BRAMs inside the fabric, we achieved maximum data-bandwidth and we were able to instantiate more DSPs that could operate in parallel and access simultaneously the weights of the model. Also, we quantized the model using fixed point arithmetic for the MAC operations so that our design synthesizes more efficiently in hardware compared with floating point arithmetic.

C. Kernel optimizations

In order to achieve large throughput we had to enable a high degree of fine-grained parallelism in application execution within the PL fabric by avoiding data dependencies. Using appropriate pragma directives we constructed a fully parallel and pipelined architecture of the Generator model with minimum latency. Also, each layer execution overlaps the next layer's operation in a dataflow fashion and intermediate results are passed down the stream without using additional memory.

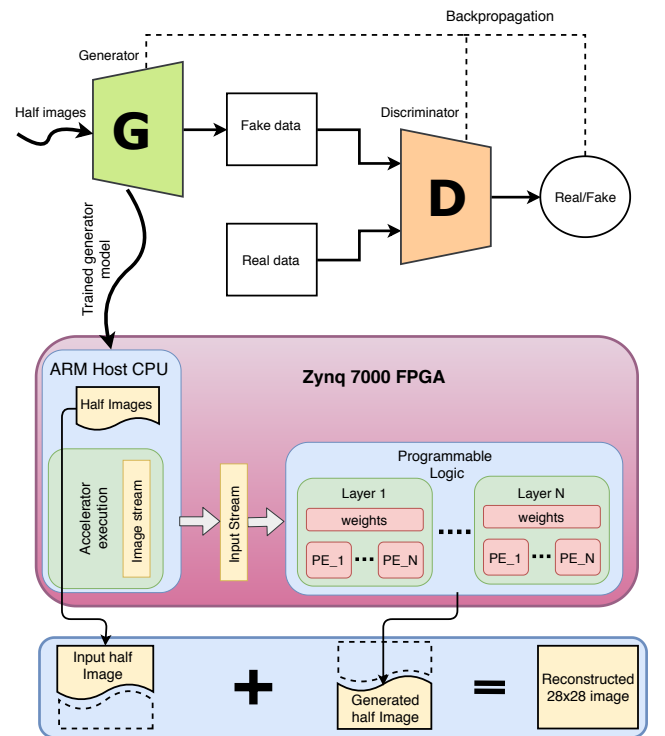


Fig. 3. Final System design

V. EVALUATION AND RESULTS

In this section we will evaluate and profile our application. We will start from the evaluation of the reconstructed images produced from the GAN model in hardware and compare the error for different bit-precisions. Also, we will report the evaluation of the hardware accelerator in terms of resources used, performance and power efficiency and compare it with other platforms.

A. Model Evaluation

We will proceed with the model evaluation. In Figure 4 we can observe the quality of the generated images of the Generator model for different fixed point precisions in hardware. As seen, the quality of the bottom half of the image that is approximated varies for different bitwidths of the multipliers in the FPGA. However, we choose to keep the 8-bit precision as it produces the best results with no significant resource overhead.

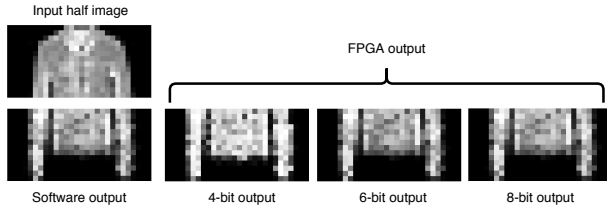


Fig. 4. Image reconstruction quality for different bit precision in the FPGA

Also, Figure 5 shows the pixel error distribution acquired from all images of the test-set (on $(-1,1)$ normalized pixel scale) for different bitwidths on the FPGA hardware. The error is compared against the software execution of the generator (32-bit float). It is also apparent from here that the 8-bit configuration performs the best overall with a *PSNR* (Peak Signal-to-Noise) of 43.14 db (on 0-255 pixel scale). Also, the *max error* acquired in the 8-bit normalized images is less than 0.1 while for less bits the max error climbs up to ~ 2 . This can result in flipping a pixel from white to black completely or vice-versa which translates to reduced quality output.

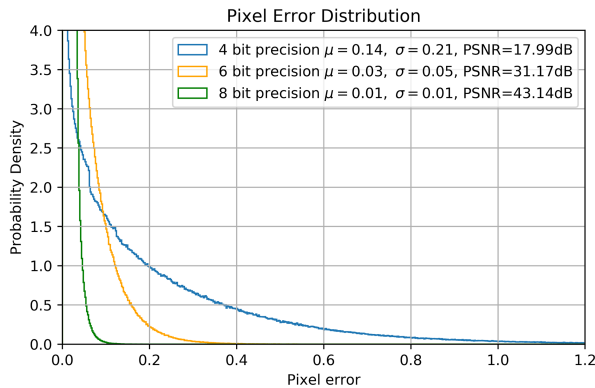


Fig. 5. Pixel error distribution for different FPGA bitwidths.

B. Accelerator Evaluation

For the system setup we used a Xilinx ZC702 which has a Zynq-7000 SoC with a Dual-core ARM Cortex-A9 and 512 MB of DDR3. Table II shows the resource utilization of our hardware accelerator on the FPGA device along with the latency timing and FPS (frames/sec) achieved.

TABLE I
RESOURCE UTILIZATION AND LATENCY OF FPGA KERNEL

Name	Utilization summary				Timing	
	BRAM	DSP	FF	LUT	Latency	FPS
Used	54	110	18907	9855	-	-
Percentage	38.57%	50%	17.77%	18.52%	0.013 (ms)	77K

C. System Evaluation

For the end-to-end evaluation of our system (memory transfers included) we deployed the same generator model on other systems (CPU, GPU) in order to compare the performance and performance/watt (PPW). The speed-up metrics are measured against the ARM A9 CPU of the FPGA SoC in single-core testing (baseline). The small-scale nature of the problem benefited the use an embedded FPGA Soc as opposed to a GPU with device overheads and produced successful results against all platforms on the performance and PPW metric.

TABLE II
PERFORMANCE AND POWER EVALUATION VS OTHER SYSTEMS

Device information		Evaluation			
System	Model	Time/Img	Speed-up	Power	PPW
CPU	ARM A9	2.06 ms	1x	3.2 W	1x
GPU	Nvidia K80	0.033 ms	62x	74 W	2.7x
FPGA	ZC702	0.013 ms	158x	3.6 W	140x

VI. CONCLUSION

In this work, we examined the use of a very promising Deep Learning algorithm called GAN for the application of image reconstruction on an FPGA. This kind of application has not been considered before for FPGA implementation. We showed the superiority of GANs compared with classical algorithms for image restoration in terms of quality but at the same time showed a successful proof-of-concept of using FPGAs with these kind of tasks with significant results in terms of accuracy, speed and power efficiency. The generator model, trained with hardware-specific optimizations, reconstructed images with high quality and also with minimal latency and power as well, compared with CPU and GPU platforms. The spectrum of possible design space tradeoffs is vast but this work shed some light to the area with successful results aiming to make FPGAs contribute fundamentally into this software-hardware ecosystem.

ACKNOWLEDGMENT

This project was funded from the Xilinx University program and the Hellenic Foundation for Research and Innovation (HFRI) and the General Secretariat for Research and Technology (GSRT), under grant agreement No 2212 - Hardware Acceleration of Machine Learning Applications in the Cloud.

REFERENCES

- [1] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial networks," 2014.
- [2] H. Alqahtani, M. Kavakli-Thorne, and D. G. Kumar Ahuja, "Applications of generative adversarial networks (gans): An updated review," *Archives of Computational Methods in Engineering*, 12 2019.
- [3] M. Ghasemzadeh, M. Samragh, and F. Koushanfar, "Rebnet: Residual binarized neural network," in *2018 IEEE 26th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, April 2018, pp. 57–64.
- [4] C. Farabet, B. Martini, P. Akselrod, S. Talay, Y. Lecun, and E. Culurciello, "Hardware accelerated convolutional neural networks for synthetic vision systems," 05 2010.
- [5] D. Danopoulos, C. Kachris, and D. Soudris, "Automatic generation of fpga kernels from open format cnn models," in *2020 IEEE 28th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, 2020, pp. 237–237.
- [6] H. Alemdar, V. Leroy, A. Prost-Boucle, and F. Pétrot, "Ternary neural networks for resource-efficient ai applications," 05 2017, pp. 2547–2554.
- [7] D. Danopoulos, C. Kachris, and D. Soudris, "Fpga acceleration of approximate knn indexing on high- dimensional vectors," in *2019 14th International Symposium on Reconfigurable Communication-centric Systems-on-Chip (ReCoSoC)*, 2019, pp. 59–65.
- [8] D. R. Mehra, "High speed ct image reconstruction using fpga," *International Journal of Computer Applications*, vol. 22, pp. 7–10, 05 2011.
- [9] S. O. Memik, A. K. Katsaggelos, and M. Sarrafzadeh, "Analysis and fpga implementation of image restoration under resource constraints," *IEEE Transactions on Computers*, vol. 52, no. 3, pp. 390–399, 2003.
- [10] S. Kumar and R. K. Jha, "An fpga-based design for a real-time image denoising using fractional integrator," *Multidimensional Systems and Signal Processing*, 02 2020.
- [11] S. Liu, C. Zeng, H. Fan, H. Ng, J. Meng, Z. Que, X. Niu, and W. Luk, "Memory-efficient architecture for accelerating generative networks on fpga," in *2018 International Conference on Field-Programmable Technology (FPT)*, 2018, pp. 30–37.
- [12] A. Yazdanbakhsh, M. Brzozowski, B. Khaleghi, S. Ghodrati, K. Samadi, N. Kim, and H. Esmailzadeh, "Flexigan: An end-to-end solution for fpga acceleration of generative adversarial networks," 04 2018, pp. 65–72.